

Indian Institute of Technology Bombay

Department of Electrical Engineering



Design and Verification of a Dual-Clock Asynchronous FIFO

Gray-Coded Pointer Crossing, Assertion-Based Verification and Hardware Validation on PYNQ-Z2

Self Project Report

Submitted by

Jyotishman Deori

Roll No: 25M1186

M.Tech. Electrical Engineering

(Electronic Systems)

Batch: 2025-27

August 2026

Abstract

This project implements a dual-clock asynchronous FIFO in Verilog-2001, for transferring data between two clock domains that have no fixed timing relationship. Pointers are Gray coded and passed through two-flop synchronisers, so that a pointer sampled by the opposite domain is never a value the counter did not actually hold.

The design was verified with a randomised scoreboard testbench under ModelSim, checked with nine SystemVerilog assertions and functional coverage under Vivado XSim, and synthesised for the Zynq XC7Z020 with no inferred latches and no warnings. It was then implemented on a PYNQ-Z2 board, with the write domain clocked from the processing system PLL and the read domain from the board's independent 125 MHz oscillator, where it transferred 286,176,459 words with zero errors while reaching both the full and empty conditions.

Contents

1	Introduction	3
1.1	Objective	3
2	Design	3
2.1	Architecture	3
2.2	Pointers and Flags	4
2.3	Module Interface	5
3	Verification	5
3.1	Simulation Testbench	5
3.2	Assertions and Coverage	5
3.3	Fault Injection	6
4	Results	7
4.1	Simulation	7
4.2	Synthesis	7
4.3	Hardware Implementation	7
5	Conclusion	9
	References	10

1. Introduction

Most digital systems contain more than one clock. A processor runs at one frequency, an interface at another, and data has to pass between them. Where two clocks have no fixed phase relationship, a signal sampled by the receiving domain may be captured while it is still changing, and the value read can be one that was never actually present.

An asynchronous FIFO is the standard structure used to cross such a boundary. It buffers data between the two domains and reports to each side whether it is safe to write or to read. The difficulty is that the full and empty conditions depend on comparing pointers maintained in different clock domains, so those pointers must themselves cross safely.

Two mechanisms are used together. Gray coding ensures that consecutive pointer values differ in exactly one bit, so a pointer sampled mid-transition returns either the old value or the new one. Two-flop synchronisers give a flip-flop that has been driven into a metastable state a full clock period to settle before anything downstream uses its output. Gray coding does not prevent metastability, and a synchroniser does not prevent multi-bit skew, so both are required.

1.1 Objective

To design a parameterised dual-clock asynchronous FIFO in Verilog, verify it in simulation with a scoreboard and with assertions, confirm through synthesis that the implementation matches the intent, and demonstrate it running on an FPGA with two independent clock sources.

2. Design

2.1 Architecture

The design consists of two files. `sync_2ff.v` is a parameterised two-flop synchroniser, kept as a separate module so that its use is explicit, and `async_fifo.v` contains the memory, the two pointer counters, the Gray conversion and the flag logic. The structure follows the standard arrangement described by Cummings [1].

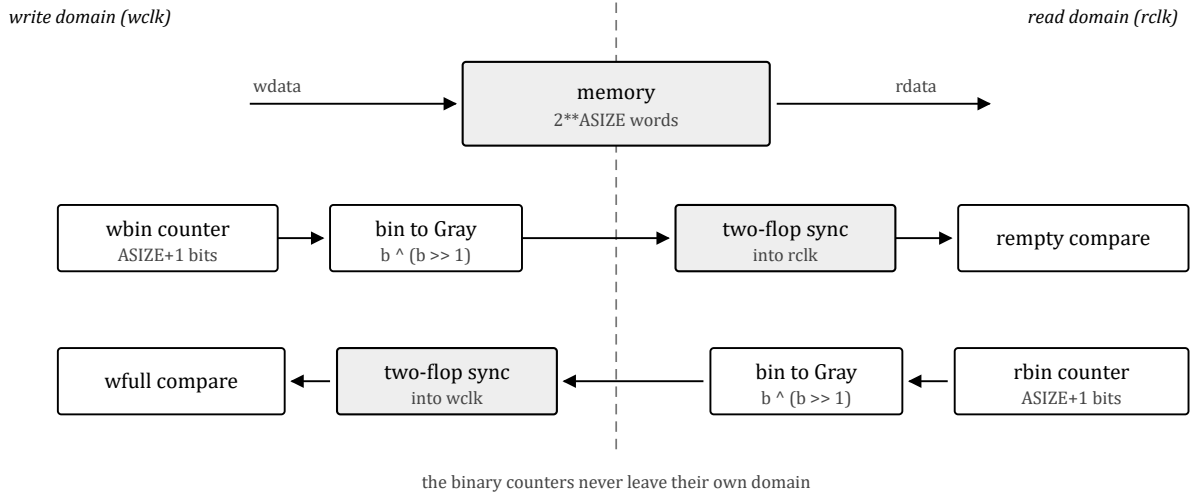


Figure 1: Block diagram of the dual-clock FIFO. Each pointer crosses once, in one direction, as a Gray-coded copy passing through a two-flop synchroniser. The binary counters themselves never cross the boundary.

2.2 Pointers and Flags

Pointers are ASIZE+1 bits wide while the memory address uses only the low ASIZE bits. The extra most-significant bit acts as a wrap flag, and it is the only thing that distinguishes the full condition from the empty condition, since the addresses are equal in both cases. Conversion to Gray code is a single exclusive-or with a right shift:

$$\text{gray} = \text{bin} \wedge (\text{bin} \gg 1) \quad (1)$$

The FIFO is empty when the read pointer has caught the write pointer with no wrap between them, so every bit of the two Gray pointers agrees:

$$\text{rempty} = (\text{rgray_next} == \text{rq2_wptr}) \quad (2)$$

The FIFO is full when the write pointer has wrapped once more than the read pointer and caught it. In Gray code this requires the low bits to agree while the top *two* bits are inverted:

$$\text{wfull} = (\text{wgray_next} == \{\sim \text{wq2_rptr}[\text{ASIZE}], \sim \text{wq2_rptr}[\text{ASIZE}-1], \text{wq2_rptr}[\text{ASIZE}-2:0]\}) \quad (3)$$

Two bits are inverted, not one. In Gray code both the top bit and the bit below it change when the counter passes the halfway point, so inverting only the most-significant bit would detect that halfway crossing rather than the wrap.

Both flags are computed from the *next* pointer value rather than the registered one, so they assert on the same clock edge on which the FIFO actually becomes full or empty. Because each domain sees a copy of the opposite pointer that is two clocks old, the FIFO may report itself full slightly after space has been freed, or empty slightly after a word has been written. The error is always in the safe direction, which is why the design cannot overflow or underflow.

2.3 Module Interface

Table 1: FIFO module interface

Signal	Dir	Domain	Function
wclk, wrst_n	in	write	Write clock and active-low asynchronous reset
winc	in	write	Write request, ignored when wfull is asserted
wdata	in	write	Write data, DSIZE bits
wfull	out	write	FIFO cannot accept a further word
rclk, rrst_n	in	read	Read clock and active-low asynchronous reset
rinc	in	read	Read request, ignored when rempty is asserted
rdata	out	read	Read data, valid whenever rempty is low
rempty	out	read	FIFO holds no unread word

The FIFO is parameterised in data width (DSIZE) and address width (ASIZE). All results reported here use an eight-bit datapath and a depth of sixteen words.

3. Verification

3.1 Simulation Testbench

A single directed test establishes little about a clock-crossing design, so the testbench stalls both interfaces pseudo-randomly and runs the two clocks at a ratio the design was not written for. Two phases execute in one simulation: a fast writer against a slow reader, at 7 ns and 11 ns, and then the reverse. Periods such as 10 ns and 20 ns were avoided because their edges align periodically, which would hide the behaviour the test is meant to exercise.

A reference queue is pushed on the write side, and popped and compared on the read side. Each phase ends by stopping the writes and reading until the FIFO is empty, so that every word written is compared on the way out. Without this drain a few hundred words remain in the FIFO at the end of a phase and are never checked.

3.2 Assertions and Coverage

ModelSim ASE supports neither SystemVerilog assertions nor covergroups, so the assertion layer runs under Vivado XSim against the same design and the same testbench. The checker is attached using `bind`, so no verification code appears in the synthesisable source. Nine properties are checked.

Table 2: Assertions checked on the design

Property	Statement
a_wgray_one_bit	Write Gray pointer changes at most one bit per cycle
a_rgray_one_bit	Read Gray pointer changes at most one bit per cycle
a_no_overflow	Occupancy never exceeds the configured depth
a_no_write_when_full	Write pointer does not advance while full
a_no_read_when_empty	Read pointer does not advance while empty
a_wptr_step	Write pointer holds or advances by exactly one
a_rptr_step	Read pointer holds or advances by exactly one
a_reset_not_full	wfull is deasserted during reset
a_reset_empty	rempty is asserted during reset

Four covergroups record whether the interesting conditions were actually reached: the write and read interfaces crossed with their request signals, the occupancy distribution including both the empty and the full bins, and concurrent activity on both interfaces. A test that passes without ever filling or emptying the FIFO demonstrates very little, so coverage is reported alongside every result.

3.3 Fault Injection

To confirm that the verification is capable of detecting a defect, the full comparison of Equation (3) was deliberately altered to invert only the most-significant bit, and the whole flow was re-run. The scoreboard and the occupancy check reported failures within 500 ns of simulation time, and under XSim the a_no_overflow assertion reported *occupancy 17 exceeds depth 16*. The correct comparison was then restored.

4. Results

4.1 Simulation

Table 3: Simulation and assertion results

Metric	ModelSim	XSim
Words written and compared	10,017	10,015
Data mismatches	0	0
Overflows / underflows	0 / 0	0 / 0
Assertion failures	not supported	0
Functional coverage	not supported	100 %
Cycles at full / at empty	12,678 / 12,419	—

Both boundary conditions were reached in quantity, which is what makes the zero mismatch count meaningful. The word counts differ slightly between the two simulators because the stimulus is randomised and their implementations of `$urandom` do not produce identical sequences from the same seed.

4.2 Synthesis

Table 4: Out-of-context synthesis results (xc7z020clg400-1)

Metric	Value	Notes
Errors / warnings / critical warnings	0 / 0 / 0	Clean elaboration
Latches inferred	0	Explicitly checked
Slice LUTs	28	20 logic, 8 distributed RAM
Slice registers	40	All flip-flops
Block RAM / DSP	0 / 0	Memory in distributed RAM

The sixteen-by-eight memory was mapped to distributed RAM rather than a block RAM, which is expected at this size. Vivado's CDC report located both crossings, each at synchroniser depth two and each carrying the `ASYNC_REG` property, confirming that the synchronisers were implemented as intended.

4.3 Hardware Implementation

The design was implemented on a PYNQ-Z2 board. A linear-feedback shift register generates the write data and an identical register on the read side predicts what should be returned, so any mismatch is counted as an error. A second pair of registers stalls both interfaces so that the full and empty

conditions are reached, and the counters are read back over JTAG.

The write domain is clocked from `FCLK_CLK0`, produced by the processing system PLL from the 33.333 MHz crystal, and the read domain from the board's separate 125 MHz oscillator. The two sources are independent, so the phase between them drifts continuously rather than repeating, which is the condition the design is intended to tolerate.

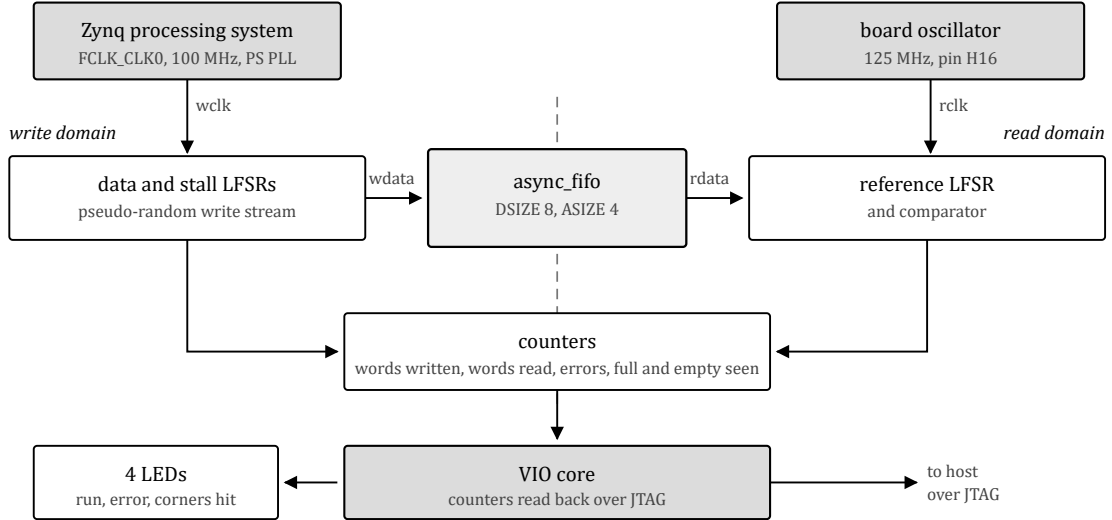


Figure 2: Hardware test setup implemented in the PYNQ-Z2 fabric. The write domain is clocked from the processing system PLL and the read domain from the board oscillator, so the two are independent. A pseudo-random stream is written into the FIFO and an identical reference sequence on the read side is compared against what emerges; any mismatch increments the error counter. Counters are read back over JTAG through a VIO core, and the LEDs give a local indication of the run.

Table 5: Hardware measurement on PYNQ-Z2, two phases of ten seconds

Phase	Words written	Words read	Errors
1 — write fast, read slow	159,905,237	159,905,221	0
2 — write slow, read fast	286,176,459	286,176,459	0
Reached full / empty			yes / yes
Worst setup slack (WNS)			1.997 ns
Worst hold slack (WHS)			0.061 ns

```

-- phase 1  write fast, read slow: w_rate 15/16, r_rate 2/16, 10 s
words written 159905237    read 159905221    errors 0
full seen 1    empty seen 1
-- phase 2  write slow, read fast: w_rate 2/16, r_rate 15/16, 10 s
words written 286176459    read 286176459    errors 0
full seen 1    empty seen 1

=====
HARDWARE RESULT    PYNQ-Z2, xc7z020clg400-1
=====

wclk    FCLK_CLK0, 100 MHz off the PS PLL
rclk    sys_clk, 125 MHz board oscillator
independent crystals, so the phase between them drifts
words written          286176459
words read+checked     286176459
data errors            0
reached full           yes
reached empty          yes

-----
PASS
=====

```

Figure 3: Console output of the hardware run, captured from the Vivado Tcl session that drove the test over JTAG. The two phases run for ten seconds each and the counters are read back once both interfaces have stopped. The full transcript is in `results/hardware_test.txt`.

Phase 1 finishes sixteen words short of the number written, which is exactly the depth of the FIFO. That phase ends with the writer running at full rate against a slow reader, so the FIFO is full when writing stops and is still holding sixteen unread words. Phase 2 reverses the rates, the reader drains those words and then keeps pace, and the two totals finish equal.

5. Conclusion

A parameterised dual-clock asynchronous FIFO was designed in Verilog-2001 and verified at three levels. In simulation it transferred 10,017 words with no mismatches across both clock ratios, with nine assertions passing and full functional coverage. Synthesis for the Zynq XC7Z020 produced no warnings and inferred no latches, and the CDC report confirmed both crossings at synchroniser depth two. On hardware, driven by two independent oscillators, it transferred 286,176,459 words with zero errors while reaching both the full and empty conditions.

The parts of the design requiring most care are the pointer width and the full comparison. The extra pointer bit is what allows full and empty to be distinguished when the addresses are identical, and the full condition requires the top two Gray bits to be inverted rather than one. Deliberately introducing an error into that comparison confirmed that the testbench and the assertions detect it, which is what gives the passing results their value.

References

- [1] C. E. Cummings, “Simulation and synthesis techniques for asynchronous FIFO design,” in *Proc. Synopsys Users Group Conference (SNUG)*, San Jose, USA, 2002.
- [2] R. Ginosar, “Metastability and synchronizers: A tutorial,” *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, Sep./Oct. 2011.
- [3] *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.
- [4] Xilinx Inc., “Zynq-7000 SoC Data Sheet: Overview (DS190),” Xilinx Technical Documentation, 2021.
- [5] AMD/Xilinx, “PYNQ: Python productivity for Zynq — framework documentation,” 2022. [Online]. Available: <http://www.pynq.io>